

Do programmer pairs make different mistakes than solo programmers?

Matthias M. Müller *

Fakultät für Informatik, Universität Karlsruhe, Am Fasanengarten 5, 76131 Karlsruhe, Germany

Received 16 June 2006; received in revised form 30 September 2006; accepted 2 October 2006

Available online 28 November 2006

Abstract

Objective: Comparison of program defects caused by programmer pairs and solo developers.

Design: Analysis of programs developed during two counter balanced experiments.

Setting: Programming lab at University.

Experimental units: 42 programs developed by computer science students participating in an extreme programming lab course.

Main outcome measures: Programmer pairs make as many algorithmic mistakes but fewer expression mistakes than solo programmers.

Results: The second result is significant on the 5% level.

Conclusions: For simple problems, pair programming seems to lead to fewer mistakes than solo programming.

© 2006 Elsevier Inc. All rights reserved.

Keywords: Pair programming; Solo programming; Defect comparison; Eureka problems

1. Introduction

Pair programming has been subject of several controlled experiments in the last years. However, the results draw a rather divergent picture of the productivity of pair programming. While Nosek (1998) and Williams et al. (2000) report a reduction of development time using pair programming Nawrocki et al. (2001) report that there is no benefit. When we look at the defect reduction capabilities, we also see a divergent picture. Williams et al. (2000) and Tomayko (2002) report on a reduced defect density caused by pair programming while Nawrocki et al. (2001) do not. And finally, according to Heiberg et al. (2003) pair programming seems to be comparable to partner programming and solo programming with reviews (see Müller, 2004, 2005).

None of the above studies investigated the impact of the complexity of the task on the productivity of pair programming. Hulkko and Abrahamsson (2005) reported that pair

programming is good for “simple problems in simple code.” However, not every code is simple and problems exist that are not easy to solve regardless of the methodology used (Knight and Leveson, 1986). On the other hand, programmer pairs form a special kind of group and groups have been studied by the social sciences for a long time. The social science developed the notion of *eureka* problems. This kind of problems have the characteristic that an individual who solves it will ordinarily feel quite certain that he is right (Steiner, 1972, p.23). The particularity of eureka problems is that the probability that the group outcome is right depends on the probability that an individual who was able to solve the problem is part of that group.

In this study, we investigated the mistakes programmer pairs and solo programmers made in a sequence of experiments (Müller, 2004, 2005). Our analysis shows that on the one hand pairs have difficulties with hard problems but on the other hand pairs expose the characteristics of groups for eureka type problems.

This paper extends the conference article (Müller, 2006) in the following topics. First, the description of the

* Tel.: +49 721 608 7333; fax: +49 721 608 7343.

E-mail address: muellerm@ira.uka.de

Polynomial task is extended to ease understanding of the presented eureka problem. Second, the comparison of the total number of defects is based only on the number of true defects. The number of maintenance activities performed by the individuals and the pairs were removed from this comparison. And finally, the defect analysis includes a correlation analysis of the total number of defects and the time needed to correct these defects.

2. Description of the experiments

The presented data was collected during two controlled experiments at the University of Karlsruhe which were conducted during the summer lectures 2002 (Exp02) and 2003 (Exp03). The experiments investigated the question whether solo programming with an additional review phase is comparable to pair programming. Participants were 38 students who took part in an Extreme Programming lab course (Müller et al., 2004). The experiments had a counter balanced design. Thus, the participants had to solve two problems: Shuffle Puzzle and Polynomial. Sections 3 and 4 describe the problems in detail. Each problem had to be solved with a different implementation method: pair programming or solo programming with review. The programming language was Java.

This section presents an overview of the experiment plan, the group sizes, and the classification process. A detailed description and the rationale for the procedures can be found in Müller (2004) or Müller (2005).

2.1. Experiment plan

The procedures for the solo programmers and the programmer pairs consist of a *Reading*, an *Implementation*, and an *Acceptance Test* phase, see Fig. 1.

Reading was identical for both procedures. During *Reading*, subjects acquainted themselves with the description of the programming task. The following two sections explain the different experiment procedures of the solo and the pair programming group.

2.1.1. Solo programming procedure

The procedure of the solo programming group is outlined by the upper half of Fig. 1. The subjects worked on their own, but they were anonymously paired off with their

pair programming partner. The solid and the dashed line in Fig. 1 represent the work flow of the code for each of the two subjects, respectively.

The *Implementation* phase of the solo programming procedure was split into *Coding*, *Review*, and *Testing*. During *Coding*, subjects had to implement the task until they thought they were done. After the two subjects finished *Coding*, they entered *Review*. During *Review*, subjects were asked to review each others program. The only purpose of the review was to find defects according to a checklist. Design flaws, violations of any sort of convention, and suggestions for a better solution were of no concern to the review. On completion of the review, subjects received the reviewed code and entered *Testing*. Subjects left *Testing* when they claimed to be done.

At this point, they entered the *Acceptance Test* phase where their programs had to pass 95% of the tests of the acceptance test. If the programs did not reach the required threshold, subjects received the output of the failed tests and had to fix the errors. The acceptance test and the subsequent rework repeated as long as the program passed less than 95% of the tests. Otherwise, the subjects concluded their work.

The presented review procedure was applied to Exp02. When following this procedure, three subjects admitted in the post-test questionnaire that they got hints for their own development from the review of the foreign program. To remove this threat in Exp03, only *one* of the two subjects, as opposed to both in Exp02, prepared the task while the other subject performed the review. The subject who was preparing the task was randomly selected. To obtain the modified experiment plan of Exp03 remove the dashed line in the solo-programming procedure of Fig. 1.

2.1.2. Pair programming procedure

The pair programming procedure was straight forward. During *Implementation*, the pairs worked on their programs until they claimed to be done. Then, they entered the *Acceptance Test* phase which also iterated between executing the acceptance test and rework. The pairs had the same exit criterion of the *Acceptance Test* phase as the solo programmers.

The pair programming procedure was the same in Exp02 and Exp03.

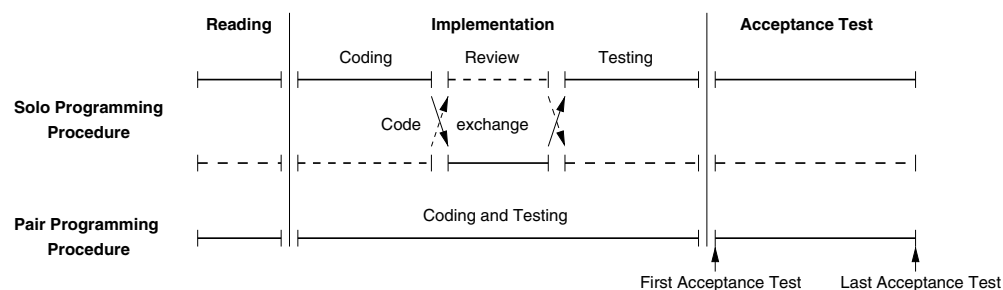


Fig. 1. Procedure for the solo and the pair programming task.

2.2. Groups and available data points

Table 1 lists the groups' task order, group sizes, and the number of available data points. Due to the counterbalanced experiment design, each subject pair is supposed to contribute one pair programming data point as well as two solo data points in the case of Exp02 and one solo data point in the case of Exp03. However, three subjects did not finish the solo task in Exp02. They belonged to the groups 2, 3, and 4, respectively. Additionally, two solo data points of group 1 and one solo data point of group 2 had to be deleted to avoid the previously mentioned internal threat of Exp02. In Exp03 all data points could be used for analysis.

Overall, there were 38 participants yielding 19 programs from programmer pairs and 23 programs from solo programmers. We have 10 data points for PP.Shu, 11 data points for So.Shu, 9 data points for PP.Pol, and 12 data points for So.Pol.

2.3. Defect classification

The purpose of the defect classification was to find all defects in the program versions right after the implementation phase. The programs are checked with test suites which contained in the case of Shuffle Puzzle 15000 and in the case of Polynomial 700000 randomly generated test cases.

To ease the defect detection process, the program versions after the acceptance-test phase were inspected first. As the exit criterion of the acceptance-test phase was to pass 95% of the tests, these programs versions were expected to contain still some defects. If a defect was found, it was removed with a minimal number of corrections and without changing the overall structure of the program. In a second step, the differences between the original program versions after the implementation and the acceptance-test phase were analysed. If a difference was found, it was decided whether it was a correction or a refactoring. Finally, each defect or refactoring was commented, labelled, and classified into one of defect categories described in the next section.

Defect classification was done by humans. The above described defect detection was done by one person. Another person inspected the defects together with their defect description and decided whether the classification

of the first person was appropriate or not. Inconsistencies or conflicts were resolved in a follow up discussion.

2.4. Defect categories

The defects were classified into one of the following categories:

Specification. These defects violate some requirements stated in the specification. For example, a correct solution to the Polynomial tasks should report the roots in ascending order. If a program did not comply with this constrain, it was counted as having a specification defect.

Expression. Defects categorised as expression defects can be fixed by the correction of a single expression or condition.

Algorithm. Algorithmic defects can be fixed by changing more than a single expression.

The last category are **Design** flaws. As opposed to the other categories, design flaws are not defects, they are refactorings (Fowler, 1999) made during the acceptance-test phase to enhance program readability. We added this kind of program changes because we assumed that the programmer pairs would develop more readable code than the solo programmers and thus, they would restructure their code less often than the solo programmers. The defect category of each defect is listed in the last columns of Tables 2 and 3.

3. Shuffle Puzzle

The first task dealt with Shuffle Puzzles, or as some like to call it, with sliding block puzzles. The problem was to find a minimal solution for a given Shuffle Puzzle within a given number of moves and to list the moves, if a solution exists. The participants had to add the method *findMoves* to the basic class *ShufflePuzzle*. Solving a Shuffle Puzzle requires implementing a backtracking algorithm using either depth-first or breadth-first search. The task description contained no algorithmic hint for a possible solution. Twenty (20) of the 21 analysed solutions used a depth-first search.

Fig. 2 shows the depth-first search part of a possible solution. The code in lines 2–4 ensures that the search does not exceed the maximum search depth. The maximum search depth is an input parameter for the implementation. The actual search depth is tracked in lines 5 and 17. The for-loop of lines 6–16 iterates through all possible moves. In general, there exist four possible directions in which pieces in a Shuffle Puzzle can be moved. However, the task description states specifically that a list of moves to a solution must not contain two identical Shuffle Puzzles. Thus, the method *allowedMoves* checks the moves performed and prevents a solution from thrashing, for example from moving one piece up and down in two subsequent moves.

Table 1
Task order and available data points for each group (PP = pair programming, So = solo, Shu = Shuffle Puzzle, Pol = Polynomial)

Group	1.Task	2.Task	Group sizes		Data pts	
			Exp02	Exp03	PP	Solo
1	PP.Shu	So.Pol	6	4	5	6
2	PP.Pol	So.Shu	4	4	4	4
3	So.Shu	PP.Pol	6	4	5	7
4	So.Pol	PP.Shu	4	6	5	6
Overall			20	18	19	23

Table 2
Classification of defects found for Shuffle Puzzle

Label	Description	Category
NoStaticFunc	Static functions removed	Design
NoCopies	Recursion does not operate on copies any more	Design
Redesign	Complete redesign of solution	Design
Data Type	Data type in breadth-first search changed	Design
Copies	Recursion operates on copies	Design
Visibility	Visibility of variables changed	Design
NoStaticVars	Search does not operate on static variables any more	Design
WrongFormat	Output has wrong format	Specification
WrongSig	Signature of interface changed	Specification
Runtime	Search was too slow	Specification
TimeConstraints	Search finishes after 15 s and returns false	Specification
WrongSolution	Solution was wrong for solved puzzles as input	Specification
Thrashing	Solution contained up–down/left–right move	Specification
NotMinimal	Provided solution was not minimal	Specification
WrongActualDepth	Actual search depth not tracked (5, 17)	Algorithm
WrongBacktrack	Actual search depth decreased two times (17)	Algorithm
NoInit	Empty field of shuffle puzzle not searched initially	Algorithm
NoDeepCopy	No deep copy of first puzzle	Algorithm
WrongSolveCond	Wrong algorithm for isSolved (8)	Algorithm
NoDeepComp	References compared instead of deep comparison (6)	Algorithm
WrongDistance	Distance from solution wrong	Algorithm
TooDeep	Search was too deep	Expression
WrongBaseCond	Recursion does not terminate correctly (2)	Expression
WrongInit	Wrong initial value for actual search depth	Expression
WrongMatrixIndex	Shuffle puzzle index calculation wrong	Expression
WrongIndex	Index calculation for empty piece wrong	Expression
WrongHistSize	Allocated space for data too small	Expression
WrongMoveDir	Directions up–down/left–right mixed up (6)	Expression
WrongSolLength	Solution has to be smaller than maximum search depth (9)	Expression
ExpThrashing	Wrong condition for recognition of thrashing (6)	Expression
Wr.OutOfBoundsCondition	Wrong identification of matrix bounds (6)	Expression

The moves are applied to the puzzle in line 7 and undone in line 15. The lines 8–12 keep track of possible solutions and store the solution if it is shorter than the shortest solution found so far. The recursive descent is performed in line 15. To ensure that the participants implemented the above described functionality of *allowedMoves*, the search had to finish within 15 s.

Table 2 lists all defects found in the Shuffle Puzzle implementations. The numbers in braces point to a line in the algorithm outlined by Fig. 2.

4. Polynomial

The task of the problem Polynomial was to find the roots of an arbitrary Polynomial of third degree. The participants had to implement the method *getZeroPositions* of the class *Polynomial*. The task description provided a hint for a possible algorithmic solution. The hint focuses on four points in the value space of the Polynomial: *lb*, *ub*, x_1 , and x_2 in Fig. 3. The first two points, the lower bound *lb* and the upper bound *ub*, mark the range where all roots of the Polynomial had to lie. These bounds are calculated with the Gershgorin Circle Theorem (Weisstein, 2006). The other two points x_1 and x_2 mark the extreme values

of the Polynomial. They are obtained by calculating the roots of the derivation. Then, the search space is divided into three intervals by using the points mentioned above. The intervals must not overlap because otherwise roots could possibly be missed. The requirement of non-overlapping intervals was not mentioned in the description, this insight was left for the participants to discover. Finally, these intervals are searched for possible roots with some fast search algorithm, for example the bisection method. Other search algorithms could be used as well. The problem of Polynomial was to identify the special cases that could occur. For example, not every Polynomial of third degree has three roots, or, an extreme value can be a root as well.

Fig. 4 shows a possible algorithm. First, the roots of the derivation have to be calculated (line 2). The quadratic formula was provided by the task description. Next, the search range has to be estimated. Afterwards, the implementation had to take into account the cases that the Polynomials can have one (lines 5–7), two (lines 8–14), or three (lines 15–28) roots. The method *isZeroPosition* (lines 9, 21, 23) identifies whether a given value represents a root or not. The method *searchZeroPosition* (lines 6, 12, 18, 20, 22, 24) searches for a root for each provided interval.

Table 3
Classification of defects found for Polynomial

Label	Description	Category
Maintenance	Maintenance	Design
DZP.ResultWrongSize	Result vector has wrong size	Specification
EPSILON.SignatureChanged	Constant ϵ changed to static	Specification
Specification.WPA	Polynomials of wrong degree accepted	Specification
Specification.NS	Roots not sorted (30)	Specification
DZP.Missed	Double root not removed from solution	Algorithm
IB.Forgotten	Missing analysis of interval bounds	Algorithm
IV.Missed	Analysis of one interval missed	Algorithm
MF.ENS	Extreme values of derivation not sorted (2)	Algorithm
Newton.WSV	Start value of newton conceptually wrong (6, 12, 18, 20, 22, 24)	Algorithm
BOUNDS.Wcalc	Bounds wrong calculated (3)	Algorithm
SPECIALCASE.OZP	Sole root not found (8–14)	Algorithm
IF.UsageWrong	Interface not used correctly	Algorithm
ALG.WrongAccuracy	Algorithm does not calculate root with proper accuracy	Algorithm
DZP.WrongComp	Wrong comparison for removal of double root	Expression
ZP.WrongIdent	Identification of root wrong (9, 21, 24)	Expression
ZP.WrongIdentInt	Identification if root exists in interval wrong	Expression
MF.DiscWrongCond	Condition for discriminant wrong (2)	Expression
MF.DiscWrongRange	Range of discriminant wrong (2)	Expression
MF.WF	Operand of derivation wrong implemented (2)	Expression
MF.WrongDerivation	Derivation in quadratic formula wrong (2)	Expression
EPSILON.Changed	ϵ changed from 10^{-4} to 10^{-9}	Expression
EPSILON.RCM	$f(x) = 0$ instead of $ f(x) < \epsilon$	Expression
Newton.WSVC	Start value of newton wrong (6, 12, 18, 20, 22, 24)	Expression
Newton.ExitCondWrong	Exit condition for newton wrong	Expression
HS.ExitCondWrong	Exit condition for bisection method wrong	Expression
BOUNDS.WrongSorted	Sorting condition for intervall bounds wrong	Expression
TYPO	Typo	Expression
PD.WrongDegree	Division of Polynomial with Polynomial of degree zero	Expression

```

1 void findMoves(Move lastMove) {
2     if ( searchDepth >= maxDepth ) {
3         return;
4     }
5     searchDepth ++;
6     for (move in allowedMoves(lastMove)) {
7         doMove(move);
8         if ( isSolved() ) {
9             if ( isMinimalSolution() ) {
10                 saveSolution();
11             }
12         } else {
13             isSolvable(move);
14         }
15         undoMove(move);
16     }
17     searchDepth --;
18     return;
19 }

```

Fig. 2. Algorithm for Shuffle Puzzle.

The search includes for example the selection of a proper start value which is essential for Newton's method. Finally, the roots should be sorted in ascending order (line 30). The

combination of the special case of an extreme value which is also a root and the floating point arithmetic was very hard to solve.

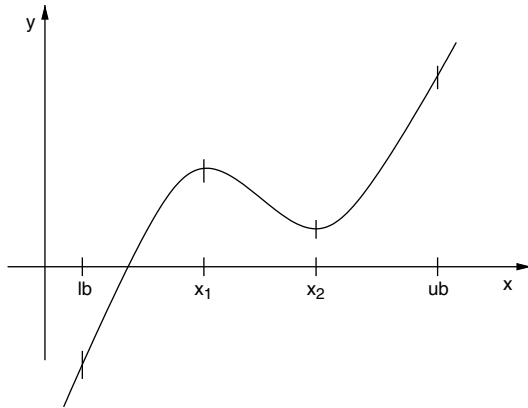


Fig. 3. Interval bounds for Polynomial.

Table 3 presents the classification of the defects found in programs of the Polynomial task.

5. Analysis of defects

The analysis of defects is divided into an analysis of the total number of defects and an analysis of the defect categories.

5.1. Analysis of total number of defects

The following comparison of the total number of defects focuses only on true defects. True defects are those defects which have been classified as Specification, Expression, or Algorithm defects. The conference paper included the number of design flaws as well. However, design flaws are not true defects, i.e. they do not cause a program failure. Thus, it seemed reasonable for us, to repeat the comparison of the number of defects using only the true defects (namely Specification, Expression, and Algorithm) and omitting the maintenance activities performed during the acceptance-test phase (design flaws). As the solo programmers performed more maintenance activities during the acceptance-test phase than the programmer pairs, the chance for the programmer pairs to outperform the individuals is smaller for the following analysis. As a consequence of the different analysis, not only the analysed data sets but the p -value of the Wilcoxon-test changed as well. However, the final result remains the same.

Fig. 5 shows the sum of the true defects caused by an experimental unit for each task. The defects of the Shuffle Puzzle programs are shown on the left hand side while the defects of the Polynomial programs are shown on the

```

1 double[] getZeroPositions() {
2     extremeValues = getExtremeValues();
3     bound = getBound();
4     switch ( extremeValues.length ) {
5     case 0:
6         result = searchZeroPosition(-bound, bound);
7         break;
8     case 1:
9         if ( isZeroPosition(extremeValues[0]) ) {
10             result = extremeValues[0];
11         } else {
12             result = searchZeroPosition(-bound, bound);
13         }
14         break;
15     case 2:
16         (eV1, eV2) = extremeValues;
17         if ( f(eV1) * f(eV2) > 0 ) {
18             result = searchZeroPosition(-bound, bound);
19         } else if ( |f(eV1)| > eps && |f(eV2)| > eps ) {
20             result = searchZeroPosition({(-bound, eV1), (eV1, eV2)},
21                                         {(eV2, bound)});
22         } else if ( isZeroPosition(eV1) ) {
23             result = (eV1, searchZeroPosition(eV2, bound));
24         } else if ( isZeroPosition(eV2) ) {
25             result = (searchZeroPosition(-bound, eV1), eV2);
26         } else {
27             result = null; // should never be reached
28         }
29         break;
30     }
31     return sort(result);
32 }
```

Fig. 4. Algorithm for Polynomial.

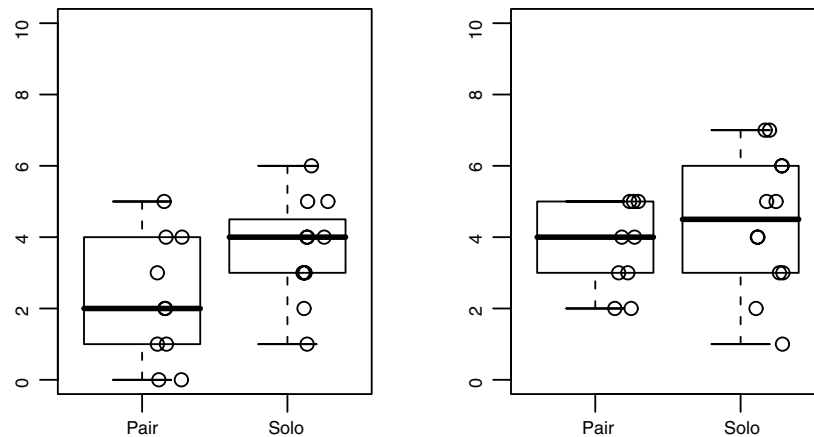


Fig. 5. Comparison of number of defects (Specification, Expression, and Algorithm) for Shuffle Puzzle (left) and Polynomial (right).

right hand side. The plot on the left hand side shows the advantage of the programmer pairs while the right hand side does not. Williams et al. (2000) suggest that programmer pairs make fewer mistakes than solo programmers. We investigated this hypothesis with the one-sided Wilcoxon test on our data set. The significance level is set to 5%. The hypothesis holds for Shuffle Puzzle ($p = 0.037$) but not for Polynomial ($p = 0.165$).

We investigated also whether the total number of defects, i.e., the sum of the Specification, Expression, and Algorithm defects, has an impact on the time spent in the acceptance-test phase. We assume that more defects lead to longer period of time spent in the acceptance-test phase. Fig. 6 shows the scatter plots for Shuffle Puzzle and Polynomial. The squares mark the data points of the pairs while the circles mark the data points of the solo programmers. The correlation of the data sets was analysed using Spearman's distribution-free test for independence (Hollander and Wolfe, 1999, 394 pp). The significance level was again set to 5%. While the data sets show a correlation for Shuffle Puzzle ($p < 0.001$), there is no dependence between the total number of defects and the time spent in the acceptance-test phase for Polynomial ($p = 0.568$).

The analysis of the number of errors led to the following results:

- The programmer pairs made significantly fewer mistakes for Shuffle Puzzle than the solo programmers. But for Polynomial, there is no difference in the number of defects between both groups.
- For Shuffle Puzzle, the time spent in the acceptance-test phase correlates with the number of defects in the program versions after the implementation phase. A similar correlation could not be found for Polynomial.

Both results are surprising. The second result points to a difference in the implementation processes of Shuffle Puzzle and Polynomial. Implementation of Shuffle Puzzle can be explained using the standard effort model for testing: more defects lead to a longer testing period. But, this explanation is not true for Polynomial. The next section sheds more light on the differences between both tasks.

5.2. Defect analysis per category

Motivated by Williams et al. (2000), we assume that for each of the above categories programmer pairs cause fewer defects than solo programmers, and, that the advantage of the programmer pairs is evenly distributed among the above categories. Fig. 7 shows the number of defects per category for Shuffle Puzzle and Polynomial. The plot on

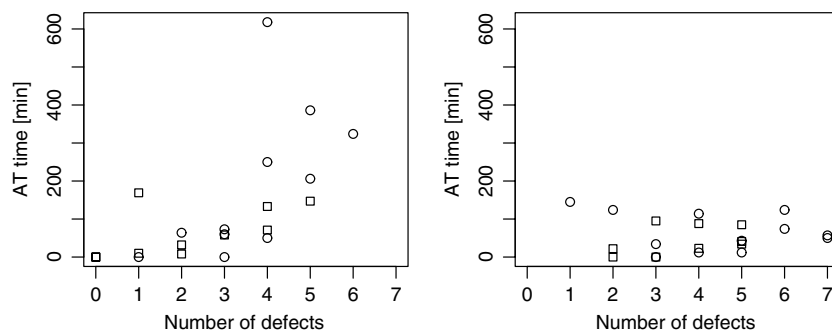


Fig. 6. Scatter plots of number of defects vs. time spent in the acceptance-test phase AT time measured in minutes for Shuffle Puzzle (left) and Polynomial (right).

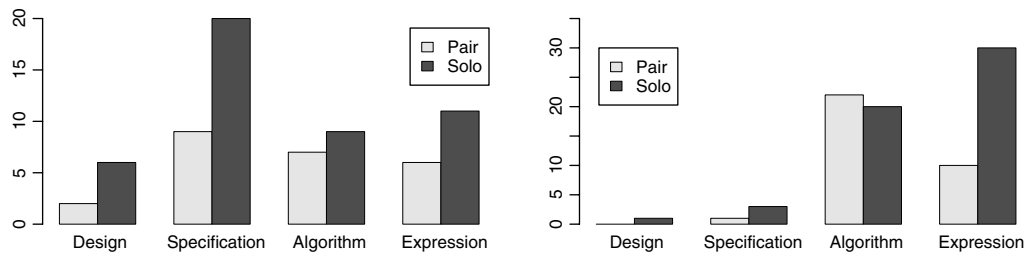


Fig. 7. Classification of defects for Shuffle Puzzle (left) and Polynomial (right).

the left hand side of Fig. 7 seems to support our expectation for the Shuffle Puzzle task. The bar of the solo programmers (the right bar in the figure) is larger for Design, Specification, Algorithm, and Expression which means that the solo programmers made more mistakes in all categories. However, the difference is smallest for the Algorithm defects. The difference is not that obvious for Polynomial, see the right plot of Fig. 7. The pairs outperform the solo programmers only for Design, Specification, and Expression, although, the advantage for Design and Specification is very small. But in contrast to Shuffle Puzzle, the pairs are not better for Algorithm defects.

Another observation concerns the relation of Algorithm to Expression defects. For Shuffle Puzzle, the pairs made only one Algorithm defect more than Expression defects. However, for Polynomial pairs made more than twice as many Algorithm defects as Expression defects.

Table 4
Wilcoxon p -values of defect classification

Task	Design	Specification	Algorithm	Expression
Shuffle Puzzle	0.18	0.02	0.50	0.15
Polynomial	0.22	0.36	0.94	0.01

Table 4 shows the p -values of the one-sided Wilcoxon test which was performed on each of the above defect classes. Differences on a 5% level are highlighted with bold values. Defects which can be fixed by correcting only a single expression seem to be reduced by pair programming regardless of the type of task. For Shuffle Puzzle, the pairs seem to make less restructurings than the solo programmers, and the solutions of the pairs adhere more to the specification than the programs of the solo developers do. This last two findings do not hold for Polynomial.

The reason why programmer pairs made fewer mistakes in the Shuffle Puzzle task as compared to the Polynomial task could be explained by the reduced number of design flaws and specification violations. But the defect classification raises another question. Why is the relation of the pairs' Algorithm defects to Expression defects larger for Polynomial than for Shuffle Puzzle? To answer this question we investigate the detailed defect distribution of the pairs and solo programmers. Figs. 8 and 9 show the detailed distribution for the Shuffle Puzzle and Polynomial task, respectively. Tables A.1 and A.2 in the appendix provide the complete data set.

The bar plot in Fig. 8 reveals that there are 12 mistakes in the Shuffle Puzzle implementations that were made by more than one pair or one solo programmer. From these

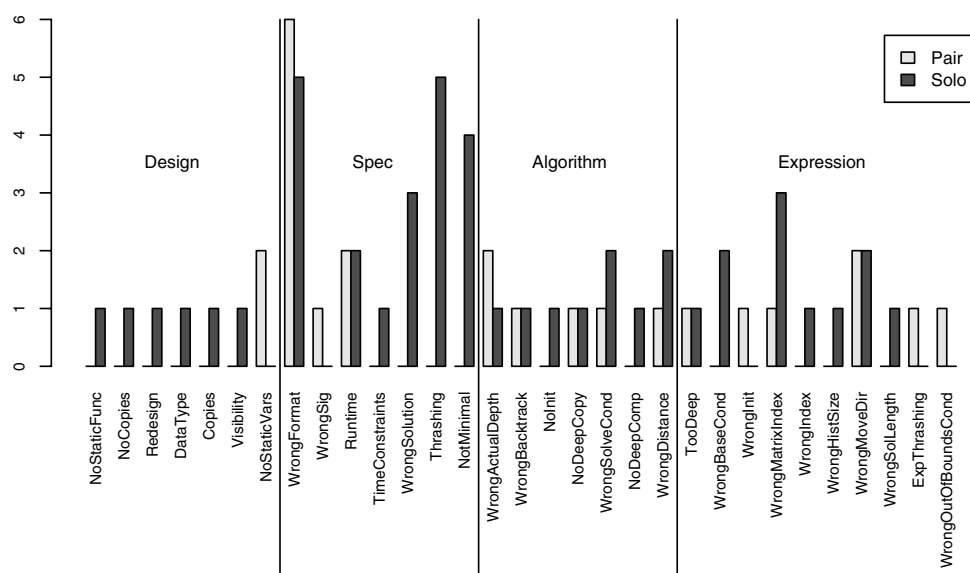


Fig. 8. Shuffle Puzzle defects in detail.

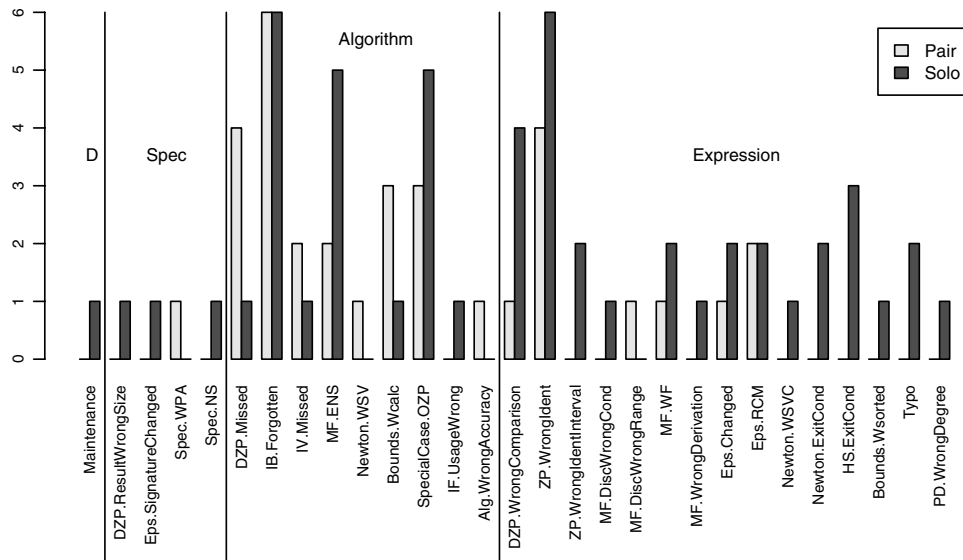


Fig. 9. Polynomial defects in detail.

12 defects, one defect belongs to Design, 5 defects belong to Specification, 3 defects belong to Algorithm, and the remaining 3 defects are Expression defects. All other defects might be seen as noise. It seems as if the complexity of the Shuffle Puzzle was the compliance to the specification.

The bar plot in Fig. 9 which shows the defect distribution for the Polynomial task reveals a different picture. Now, 15 defects were made by more than one pair or solo programmer. From these 15 defects, 6 defects belong to Algorithm and the other 9 defects belong to Expression. In contrast to Shuffle Puzzle, Polynomial was algorithmically more challenging while adherence to the specification was a minor problem.

6. Why were there differences?

After we had analysed the defects, we were interested in the reasons for the differences between the Shuffle Puzzle and Polynomial defect distributions.

6.1. Specification defects

First, we concentrate on the Specification defects of the Shuffle Puzzle task. The main problem of this task was to comply to the specification. The implementation problem consisted of a simple backtracking algorithm to which the students have known the solution since their first computer science courses. Thus, reading the task description could have led to premature conclusions what the task is about. The reason why pairs made fewer Specification defects could be their larger collective memory (Forsyth, 2006, p. 332) which reduces the chance that the pair misses a requirement stated in the specification as compared to a solo programmer. However, the larger collective memory did not prevent the pairs from violating the output format.

Six pairs violated the specification of the output format while only 5 solo developers did not comply to this requirement as well, see defect *WrongFormat*.

6.2. Algorithm defects

The next topic focuses on the algorithmic defects in the Polynomial programs as compared to the Shuffle Puzzle programs. The Polynomial task was more complicated than the Shuffle Puzzle task. For example, the defect *IB.Forgetting* presented by Fig. 9 was made by 6 independent solo developers and by 6 independent pairs. The fact that interval bounds have to be inspected in order to calculate the roots correctly is hard to identify. As Knight and Leveson (1986) identify problems which are hard to solve regardless of the used methodology, it is reasonable that these problems are hard to solve for programmer pairs as well.

6.3. Eureka problems

There are defects apart from the above discussed Specification defects which were introduced by fewer pairs than solo programmers. This difference can be partially explained with *eureka* problems from the social psychology. For an eureka problem, an individual who solves it will ordinarily feel quite certain that he is right. A typical eureka problem is for example the following one:

On one side of a river are three wives and three husbands. All of the men but none of the woman can row. Get them all across the river by means of a boat carrying only three at one time. No man will allow his wife to be in the presence of another man unless he is also there (Steiner, 1972, p. 19).

In the terms of Steiner's taxonomy of tasks (see Steiner, 1972 or Forsyth, 2006, 309 pp.), the above cited problem is *unitary*, *optimizing*, and *disjunctive*. A problem is said to be *unitary* if it is not divisible into subcomponents. *Optimizing* problems need a correct or an optimal solution. And for *disjunctive* problems the group selects one solution from a pool of members' solutions.

The characteristic of this wives and husbands problem is that the proper solution is readily apparent or easily demonstrated once that solution has been obtained. This characteristic of eureka problems leads to the observation that groups find the right solution as long as one of its individuals possesses the ability to solve the problem. Thus, if the probability that one individual from a group of individuals fails to solve the problems is Q , the probability that a group made up of two individuals fails to solve the problem as well reduces to Q^2 . We call this effect the eureka pattern. This pattern can be seen at its best for the defect *MF.ENS*. *MF.ENS* states that the participants forgot to sort the roots of the derivation, denoted as x_1 and x_2 in Fig. 3. Thus, the three intervals which divide the search space for possible roots of the Polynomial are not necessary disjoint. But if the intervals overlap some roots may be calculated twice and others are missed completely. If you are pointed to the solution, that sorted values are necessary for a correct calculation of the interval bounds, you will be convinced directly. Thus, *MF.ENS* is an eureka problem. From the 12 solo programmers 11 had the chance to make this mistake because one programmer used a different algorithm. Now, 5 out of 11 developers made this error, see Fig. 9, leading to a theoretical failure probability for the pairs of $(5/11)^2 = 0.21$ which is close to the actual failure probability of $2/9 = 0.22$. Similar values can be calculated for another eureka problem, for example *WrongMatrixIndex* in Shuffle Puzzle with $(3/11)^2 = 0.07$ and $1/10 = 0.1$.

The discussion of the effects of eureka problems is interesting as it points to problem domains where the usage of programmer pairs instead of solo programmers might lead to a reduced risk of failure. For example, if a technological problem has to be solved and the solution can easily identified as the right one, the usage of a programmer pair does not necessarily mean nearly doubled personnel cost but rather a reduced probability of failure.

However, there still exist some defects where the number of pairs who made this defect exceeds the number of solo developers, e.g., *Bounds.Wcalc*, *DZP.Missed*, *IV.Missed*, and *WrongActualDepth*. So far, we do not know whether these failures are only due to chance or whether they expose another characteristic of the pair programming situation because these defects have been classified as being Algorithm defects.

6.4. General issues

Our own identification of eureka problems started with the observation that the distribution of some defects follow the eureka pattern. We then read the description of the

defects in question and decided that the problems which caused these defects were eureka problems.

Now, we ask whether there are other defects which point to a possible eureka problem but whose defect distributions among the pairs and solo programmers do not follow the eureka pattern. During the inspection of the defect descriptions we came to the conclusion that for us, every defect was caused by an eureka problem, although, only a small number of the defect distributions follow the eureka pattern. We concluded, that the identification whether a problem is an eureka problem or not mainly depends on the skills to understand the problem and the solution to it. Thus, whether a defect distribution in this study shows the eureka pattern might depend on the educational background of the participating students: If the students did not possess the right skills to understand the problem and its solution then it might be unlikely that two students as a programmer pair perform better than a student who works on the same problem on his own. The small number of Specification defects for Shuffle Puzzle made by the pairs seems to support this assumption because specification questions can be easily resolved by pointing to the appropriate paragraph of the specification. Another support for our assumptions are those defects that were inserted by more pairs than solo programmers. These defects are all Algorithm defects which might be too complicated to be understandable by our students.

To sum up the previous paragraph, we assume that whether a problem is an eureka problem or not depends on the skills of the individuals of a group working on this problem. Now, we try to generalise this assumption to an industrial setting: If our assumption is true, pair programming shows the largest advantage for recurring problems because the solutions for recurring problems are known by the developers and the possibility that it is found is larger for a pair than for an individual programmer. This assumption is supported by the statement of Hulkko and Abrahamsson (2005) where pair programming is seen most beneficial for simple problems in simple code.

7. Threats to validity

Defect identification was done by a single person. But as the classification was reviewed by a second person the possibility of a bias within the data set is reduced.

Participants were students and the pairs were ad-hoc pairs. Thus, the presented figures might be different if professionals and harmonising pairs had been used instead.

As the discussion in the previous section pointed out, the data set of this study depends on the educational background of our students. As a result, the analysed defect distributions might be different if the participants had different programming skills or a different educational background.

At last, we look at the original purpose of the experiments the data was drawn from. In these experiments, solo programmers had an additional review of their program

Table A.2

Defects inserted by experiment unit for the Polynomial task (P = pair, S = solo)

PNR	Method	Alg. Wrong Accuracy	Bounds. Wcalc	Bounds. Wsorted	DZP. Missed	DZP. Result WrongSize	DZP. WrongComparison	Epsilon. Changed	Epsilon. RCM	Epsilon. SignatureChanged	HS.ExitCond	IB. Forgotten	IF. UsageWrong	IV. Missed	Maintenance	MF.DiscWrongCond	MF. DiscWrongRange	MF.ENS	MF.WF	MF. WrongDerivation	Newton.ExitCond	Newton. WSV	Newton. WSVC	PD. WrongDegree	SpecialCase. OZP	Specification. NS	Specification. WPA	Typo	ZP. WrongIdentInterval	ZP. WrongIdent
101	P	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
102	P	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
402	S	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
403	S	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
404	S	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
406	S	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
501	P	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	
502	P	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
503	P	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
801	S	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
803	S	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
804	S	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
2101	P	0	0	0	1	0	0	1	0	0	0	1	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	1	
2102	P	0	1	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
2402	P	0	1	0	0	0	0	0	1	0	0	1	0	1	0	0	0	1	0	0	0	0	0	0	1	0	0	0	0	
2404	P	0	0	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
2501	S	0	0	0	0	0	1	0	1	0	0	1	1	0	0	0	0	0	1	0	0	1	0	0	0	0	0	1	0	
2502	S	0	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	
2802	S	0	0	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	
2803	S	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
2805	S	0	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	

References

- Forsyth, D., 2006. Group Dynamics, Fourth ed. Thomson Wadsworth.
- Fowler, M., 1999. Refactoring: Improving the Design of Existing Code. Addison Wesley.
- Heiberg, S., Puus, U., Salumaa, P., Seeba, A., 2003. Pair-programming effect on developers productivity. In: XP 2003LNCS, vol. 2675. Springer, pp. 215–224.
- Hollander, M., Wolfe, D., 1999. Nonparametric Statistical Methods, Second ed. John Wiley & Sons.
- Hulkko, H., Abrahamsson, P., 2005. A multiple case study on the impact of pair programming on product quality. In: International Conference on Software Engineering. St. Louis, Missouri, USA, pp. 495–503.
- Knight, C., Leveson, N., 1986. An experimental evaluation of the assumption of independence in multi-version programming. IEEE Transactions on Software Engineering 12 (1), 96–109.
- Müller, M., 2004. Are reviews an alternative to pair Programming? Journal on Empirical Software Engineering 9 (4), 335–351.
- Müller, M., 2005. Two controlled experiments concerning the comparison of pair programming to peer review. Journal of Systems and Software 78, 166–179.
- Müller, M., 2006. Do programmer pairs make different mistakes than solo programmers? In: Evaluation and Assessment in Software Engineering (EASE). Keele, UK, pp. 69–78.
- Müller, M., Link, J., Sand, R., Malpohl, G., 2004. Extreme programming in curriculum: Experiences from academia and industry. In: Conference on Extreme Programming and Agile Processes in Software Engineering (XP2004). Garmisch-Partenkirchen, Germany, pp. 294–302.
- Nawrocki, J., Wojciechowski, A., 2001. Experimental evaluation of pair programming. In: European Software Control and Metrics (Escom). London, UK.
- Nosek, J., 1998. The case for collaborative programming. Communications of the ACM 41 (3), 105–108.
- Steiner, I., 1972. Group Process and Productivity. Academic Press.
- Tomayko, J., 2002. A comparison of pair programming to inspections for software defect reduction. Computer Science Education 12 (3), 213–222.
- Weinstein, E., 2006. Gershgorin circle theorem. From MathWorld—A Wolfram Web Resource. <http://mathworld.wolfram.com/GershgorinCircleTheorem.html>.
- Williams, L., Kessler, R., Cunningham, W., Jeffries, R., 2000. Strengthening the case for pair-programming. IEEE Software, 19–25.

Matthias Müller received the diploma and PhD degrees in informatics from the University of Karlsruhe, Germany, in 1996 and 2000. In his dissertation, he worked on the topic of optimizing compilers for parallel architectures. In the last years, he has focused on software process improvement, especially lightweight software processes. In particular, he works on the empirical and economical evaluation of the techniques proposed by extreme programming.